Relatório — Simulador de Resolução de Equações Algébricas e Transcendentes

Cálculo Numérico — UNIVASF 2026.2 — Projeto Unidade 1 (Zeros) Equipe: Giovanni Éber, Caio Benevides, Tãua Oliveira Entrega do PDF: 16/10/2026

> Esqueleto: os números das tabelas vieram de uma execução real (eps = 1e-6, max_iter = 100). **Os tempos mudam a cada execução**; rodem de novo no computador final e atualizem antes de gerar o PDF. Trechos `[PREENCHER]` dependem de vocês.

---

1. Descrição do programa e como executá-lo

- Objetivo: comparar Bissecção, Falsa Posição, Newton-Raphson e Secante (raiz, iterações, tempo e |f(raiz)|).
- Linguagem e bibliotecas: Python, `sympy` (leitura da função e derivada automática) e `matplotlib` (gráficos e animações).
- Execução:

```
pip install -r requirements.txt
python main.py      # menu
python testes.py    # 36 testes dos métodos
```

- Funcionalidades: casos prontos, função personalizada, eps e max_iter configuráveis, tabela no terminal, janela com gráficos, animação das iterações, opção de novo cálculo ou sair.
- Tratamento de falhas: sem mudança de sinal, divisão por zero, derivada nula, saída do intervalo (aviso, sem abortar), entrada inválida.
- Critério de parada igual nos 4 métodos: $|f(x)| < eps$.
- Estrutura dos arquivos: ver tabela do README. [PREENCHER: figura `docs/fluxo_dados.png` opcional]

2. Resultados

Parâmetros em todos os casos: $\varepsilon = 10^{-6}$, máximo de 100 iterações.

2.1 Exemplo de teste: f(x) = cos(x) − x

Dados: [a, b] = [0, 1]; Newton $x_0 = 0{,}5$, $f'(x) = -\text{sen}(x) - 1$; Secante $x_0 = 0$, $x_1 = 1$. Raiz de referência: 0,7390851332.

Método	Raiz	Iterações	Tempo (ms)	|f(raiz)|
Bissecção	0,7390851974	20	0,0200	1,08e-07

Método	Raiz	Iterações	Tempo (ms)	|f(raiz)|
Falsa Posição	0,7390847824	5	0,0060	5,87e-07
Newton-Raphson	0,7390851339	3	0,0113	1,18e-09
Secante	0,7390851121	4	0,0050	3,53e-08

2.2 Problema 1 — Resfriamento: f(t) = 20 + 90e^(−0,15t) − 50

Dados: [a, b] = [0, 20] (f(0) = 60 > 0, f(20) ≈ −25,5 < 0); Newton $t_0 = 0$, f′(t) = −13,5e^(−0,15t); Secante $t_0 = 0$, $t_1 = 20$. Raiz exata: ln(3)/0,15 ≈ 7,3240819245.

Método	Raiz	Iterações	Tempo (ms)	|f(raiz)|
Bissecção	7,3240819573	26	0,0164	1,48e-07
Falsa Posição	7,3240821223	23	0,0133	8,90e-07
Newton-Raphson	7,3240819244	5	0,0063	3,97e-10
Secante	7,3240819245	10	0,0098	4,33e-11

Observação: a Secante **saiu do intervalo** na iteração 2 (t = −3,4757, fora de t ≥ 0) e mesmo assim convergiu. O programa registra isso como aviso, não como falha.

2.3 Problema 2 — Capacitor RC: f(t) = 5(1 − e^(−t)) − 3,8

Dados: [a, b] = [0, 3] (f(0) = −3,8 < 0, f(3) ≈ 0,95 > 0); Newton $t_0 = 0$, f′(t) = 5e^(−t); Secante $t_0 = 0$, $t_1 = 3$. Raiz exata: −ln(0,24) ≈ 1,4271163556.

Método	Raiz	Iterações	Tempo (ms)	|f(raiz)|
Bissecção	1,4271168709	21	0,0126	6,18e-07
Falsa Posição	1,4271170326	25	0,0116	8,12e-07
Newton-Raphson	1,4271163488	5	0,0056	8,18e-09
Secante	1,4271163561	9	0,0061	5,55e-10

[PREENCHER: prints da janela de gráficos de cada caso, `docs/` ou captura nova]

2.4 Falhas observadas e testadas

Situação	Método	Resultado
Sem mudança de sinal em [a, b]	Bissecção, Falsa Posição	erro informado
Derivada nula ($x_0 = 0$)	Newton-Raphson	erro informado
$f(x_1) = f(x_0)$	Secante	erro informado (divisão por zero)
Saída do intervalo	Newton, Secante	aviso, continua

[PREENCHER: função e dados usados em cada teste de falha, vistos em `testes.py`]

3. Análise crítica

Pontos a desenvolver:

1. **Métodos de intervalo × abertos.** Bissecção e Falsa Posição precisaram de 20 a 26 iterações (exceto Falsa Posição no exemplo, 5); Newton precisou de 3 a 5 e a Secante de 4 a 10. Explicar pela ordem de convergência: linear, quadrática (Newton) e superlinear (Secante, ≈ 1,618).
2. **Garantia × velocidade.** Intervalo sempre converge se há mudança de sinal, mas é lento; abertos são rápidos, mas dependem do chute inicial e da derivada.
3. **Falsa Posição.** Foi a mais rápida entre as de intervalo no exemplo (5 iterações) e a mais lenta no Problema 2 (25). [PREENCHER: explicar com a curvatura da função e a extremidade que fica "presa"]
4. **Qual foi mais eficiente em cada caso:** Newton em todos, em iterações. [PREENCHER: comentar que ele exige f', calculada aqui por sympy]
5. **Resíduo × iterações.** A Bissecção termina com |f| bem menor que eps em alguns casos (1,08e-07) por parar em um passo de bisseção, não por ajuste fino.
6. **Tempos.** Todos na ordem de centésimos de ms; diferenças pequenas e sujeitas a ruído. Priorizar iterações e resíduo na comparação.
7. **Dificuldades.** [PREENCHER: escolha de intervalos e estimativas iniciais, saída do intervalo na Secante do Problema 1, critério de parada único, etc.]
8. **Relevância do modelo RC** (pedido no Problema 2): circuitos de temporização, sensores e sistemas embarcados. [PREENCHER: o instante em que V = 3,8 V define quando um limiar de tensão é atingido, ou seja, um tempo de disparo ou de atraso]

4. Conclusão

[PREENCHER: equipe toda. Resumir qual método recomendar e quando.]

5. Participação de cada membro

- **Giovanni Éber** (Parte 3): menu, entrada de dados, tabela, gráficos, animações. [PREENCHER]
- **Caio Benevides** (Parte 1): Bissecção e Falsa Posição, tratamento de ausência de mudança de sinal e divisão por zero. [PREENCHER]
- **Tãua Oliveira** (Parte 2): Newton-Raphson e Secante, tratamento de derivada nula, divisão por zero e saída do intervalo. [PREENCHER]

Checklist antes de gerar o PDF

- ☐ Atualizar os tempos com uma execução final
- ☐ Inserir os gráficos de cada caso
- ☐ Preencher todos os `[PREENCHER]`
- ☐ Conferir o link do repositório no GitHub
- ☐ Exportar em PDF e enviar até 16/10/2026